

Lab 5: Linear Regression through Gradient Descent

Aim: To implement Linear Regression using the Gradient Descent optimization algorithm and evaluate its performance on the Student Performance dataset.

Dataset: Student Performance Dataset (UCI Machine Learning Repository)

Target Variable: G3 (Final Grade, 0-20)

1. Import Libraries

```
In [ ]: # Final comprehensive dashboard
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Self-Learning Component Dashboard', fontsize=16, fontweight='bold')

# 1. Cost convergence comparison
axes[0, 0].plot(model.cost_history, label='Basic GD', linewidth=2)
axes[0, 0].plot(sl_model.cost_history, label='Self-Learning GD', linewidth=2)
axes[0, 0].set_title('Cost Convergence')
axes[0, 0].set_xlabel('Iterations')
axes[0, 0].set_ylabel('Cost')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# 2. Actual vs Predicted (Self-Learning)
axes[0, 1].scatter(y_test, y_pred_sl, alpha=0.6, color='steelblue', edgecolor='k')
axes[0, 1].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
                'r--', linewidth=2, label='Ideal')
axes[0, 1].set_title('Self-Learning: Actual vs Predicted')
axes[0, 1].set_xlabel('Actual G3')
axes[0, 1].set_ylabel('Predicted G3')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# 3. Metrics bar chart comparison
metrics_names = ['MAE', 'MSE', 'RMSE']
basic_vals = [mae, mse, rmse]
sl_vals = [mae_sl, mse_sl, rmse_sl]
x = np.arange(len(metrics_names))
width = 0.3
axes[1, 0].bar(x - width/2, basic_vals, width, label='Basic GD', color='coral',
               edgecolor='black')
axes[1, 0].bar(x + width/2, sl_vals, width, label='Self-Learning GD', color='mediumslateblue',
               edgecolor='black')
axes[1, 0].set_xticks(x)
axes[1, 0].set_xticklabels(metrics_names)
axes[1, 0].set_title('Error Metrics Comparison')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3, axis='y')

# 4. R2 Score comparison
r2_vals = [r2, r2_sl]
```

```
bars = axes[1, 1].bar(['Basic GD', 'Self-Learning GD'], r2_vals,
                      color=['coral', 'mediumseagreen'], edgecolor='black', width
for bar, val in zip(bars, r2_vals):
    axes[1, 1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
                    f'{val:.4f}', ha='center', fontweight='bold')
axes[1, 1].set_title('R2 Score Comparison')
axes[1, 1].set_ylabel('R2 Score')
axes[1, 1].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()
```

All libraries imported successfully.

2. Load the Dataset

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import warnings
warnings.filterwarnings('ignore')

print("All libraries imported successfully.")
```

Dataset shape: (395, 33)

Number of samples: 395

Number of features: 33

```
Out[ ]: 
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	fan
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	
3	GP	F	15	U	GT3	T	4	2	health	services	...	
4	GP	F	16	U	GT3	T	3	3	other	other	...	

5 rows × 33 columns



3. Exploratory Data Analysis

```
In [63]: # Basic information about the dataset
print("=" * 50)
print("DATASET INFO")
print("=" * 50)
df.info()

print("\n" + "=" * 50)
print("STATISTICAL SUMMARY")
```

```
print("=" * 50)
df.describe()
```

```
=====
DATASET INFO
=====
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
#   Column      Non-Null Count  Dtype
---  -
0   school      395 non-null    object
1   sex         395 non-null    object
2   age         395 non-null    int64
3   address     395 non-null    object
4   famsize     395 non-null    object
5   Pstatus     395 non-null    object
6   Medu        395 non-null    int64
7   Fedu        395 non-null    int64
8   Mjob        395 non-null    object
9   Fjob        395 non-null    object
10  reason      395 non-null    object
11  guardian    395 non-null    object
12  traveltime  395 non-null    int64
13  studytime   395 non-null    int64
14  failures    395 non-null    int64
15  schoolsup   395 non-null    object
16  famsup      395 non-null    object
17  paid        395 non-null    object
18  activities  395 non-null    object
19  nursery     395 non-null    object
20  higher      395 non-null    object
21  internet    395 non-null    object
22  romantic    395 non-null    object
23  famrel      395 non-null    int64
24  freetime    395 non-null    int64
25  goout       395 non-null    int64
26  Dalc        395 non-null    int64
27  Walc        395 non-null    int64
28  health      395 non-null    int64
29  absences    395 non-null    int64
30  G1          395 non-null    int64
31  G2          395 non-null    int64
32  G3          395 non-null    int64
dtypes: int64(16), object(17)
memory usage: 102.0+ KB
```

```
=====
STATISTICAL SUMMARY
=====
```

Out[63]:

	age	Medu	Fedu	travelttime	studytime	failures	fam
count	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000
mean	16.696203	2.749367	2.521519	1.448101	2.035443	0.334177	3.944300
std	1.276043	1.094735	1.088201	0.697505	0.839240	0.743651	0.896000
min	15.000000	0.000000	0.000000	1.000000	1.000000	0.000000	1.000000
25%	16.000000	2.000000	2.000000	1.000000	1.000000	0.000000	4.000000
50%	17.000000	3.000000	2.000000	1.000000	2.000000	0.000000	4.000000
75%	18.000000	4.000000	3.000000	2.000000	2.000000	0.000000	5.000000
max	22.000000	4.000000	4.000000	4.000000	4.000000	3.000000	5.000000



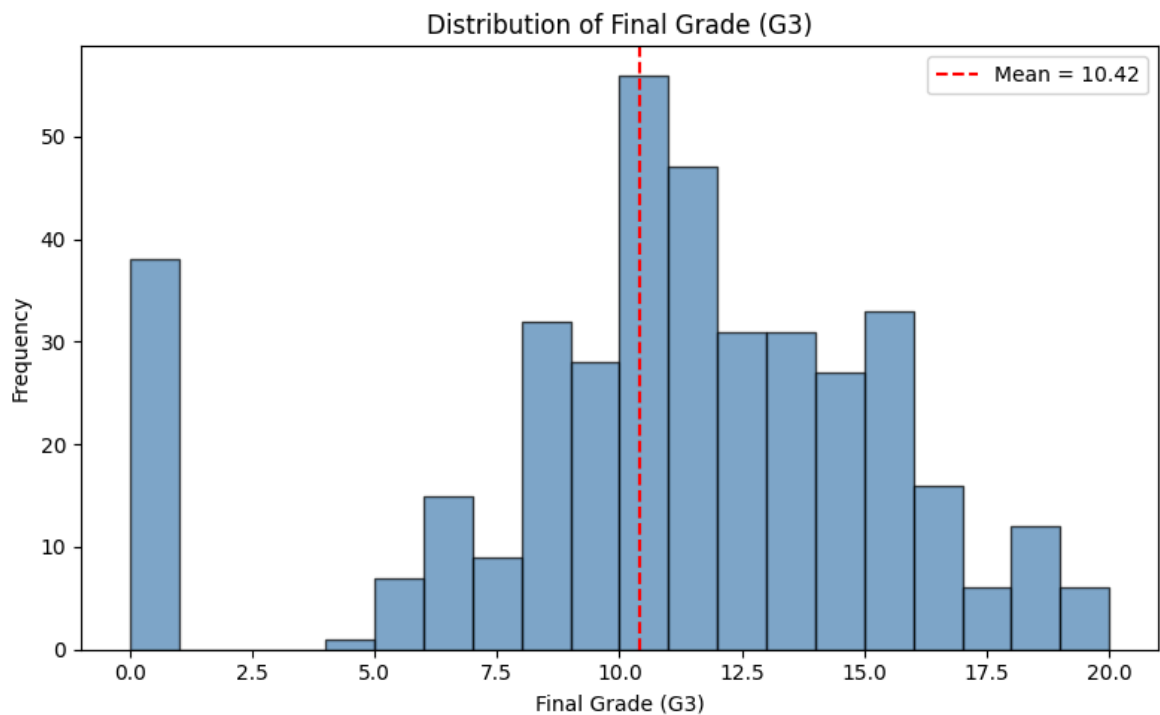
```
In [64]: # Check for missing values
print("Missing values per column:")
missing = df.isnull().sum()
print(missing[missing > 0] if missing.sum() > 0 else "No missing values found.")
print(f"\nTotal missing values: {df.isnull().sum().sum()}")
```

Missing values per column:

No missing values found.

Total missing values: 0

```
In [65]: # Distribution of the target variable (G3 - Final Grade)
plt.figure(figsize=(8, 5))
plt.hist(df['G3'], bins=20, edgecolor='black', color='steelblue', alpha=0.7)
plt.xlabel('Final Grade (G3)')
plt.ylabel('Frequency')
plt.title('Distribution of Final Grade (G3)')
plt.axvline(df['G3'].mean(), color='red', linestyle='--', label=f"Mean = {df['G3'].mean():.2f}")
plt.legend()
plt.tight_layout()
plt.show()
```



```
In [66]: # Identify categorical and numerical columns
categorical_cols = df.select_dtypes(include='object').columns.tolist()
numerical_cols = df.select_dtypes(include=np.number).columns.tolist()

print(f"Categorical columns ({len(categorical_cols)}): {categorical_cols}")
print(f"\nNumerical columns ({len(numerical_cols)}): {numerical_cols}")
```

Categorical columns (17): ['school', 'sex', 'address', 'famsize', 'Pstatus', 'Mjob', 'Fjob', 'reason', 'guardian', 'schoolsup', 'famsup', 'paid', 'activities', 'nursery', 'higher', 'internet', 'romantic']

Numerical columns (16): ['age', 'Medu', 'Fedu', 'traveltime', 'studytime', 'failures', 'famrel', 'freetime', 'goout', 'Dalc', 'Walc', 'health', 'absences', 'G1', 'G2', 'G3']

4. Data Preprocessing

```
In [67]: # Encode categorical variables using Label Encoding
df_encoded = df.copy()
label_encoders = {}

for col in categorical_cols:
    le = LabelEncoder()
    df_encoded[col] = le.fit_transform(df_encoded[col])
    label_encoders[col] = le
    print(f"{col}: {dict(zip(le.classes_, le.transform(le.classes_)))}")

print("\nEncoding complete.")
df_encoded.head()
```

```

school: {'GP': np.int64(0), 'MS': np.int64(1)}
sex: {'F': np.int64(0), 'M': np.int64(1)}
address: {'R': np.int64(0), 'U': np.int64(1)}
famsize: {'GT3': np.int64(0), 'LE3': np.int64(1)}
Pstatus: {'A': np.int64(0), 'T': np.int64(1)}
Mjob: {'at_home': np.int64(0), 'health': np.int64(1), 'other': np.int64(2), 'serv
ices': np.int64(3), 'teacher': np.int64(4)}
Fjob: {'at_home': np.int64(0), 'health': np.int64(1), 'other': np.int64(2), 'serv
ices': np.int64(3), 'teacher': np.int64(4)}
reason: {'course': np.int64(0), 'home': np.int64(1), 'other': np.int64(2), 'reput
ation': np.int64(3)}
guardian: {'father': np.int64(0), 'mother': np.int64(1), 'other': np.int64(2)}
schoolsup: {'no': np.int64(0), 'yes': np.int64(1)}
famsup: {'no': np.int64(0), 'yes': np.int64(1)}
paid: {'no': np.int64(0), 'yes': np.int64(1)}
activities: {'no': np.int64(0), 'yes': np.int64(1)}
nursery: {'no': np.int64(0), 'yes': np.int64(1)}
higher: {'no': np.int64(0), 'yes': np.int64(1)}
internet: {'no': np.int64(0), 'yes': np.int64(1)}
romantic: {'no': np.int64(0), 'yes': np.int64(1)}

```

Encoding complete.

Out[67]:

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel
0	0	0	18	1	0	0	4	4	0	4	...	4
1	0	0	17	1	0	1	1	1	0	2	...	5
2	0	0	15	1	1	1	1	1	0	2	...	4
3	0	0	15	1	0	1	4	2	1	3	...	3
4	0	0	16	1	0	1	3	3	2	2	...	4

5 rows × 33 columns



In [68]:

```

# Correlation heatmap of numerical features with G3
correlation = df_encoded.corr()['G3'].sort_values(ascending=False)
print("Correlation of features with G3 (Final Grade):")
print(correlation)

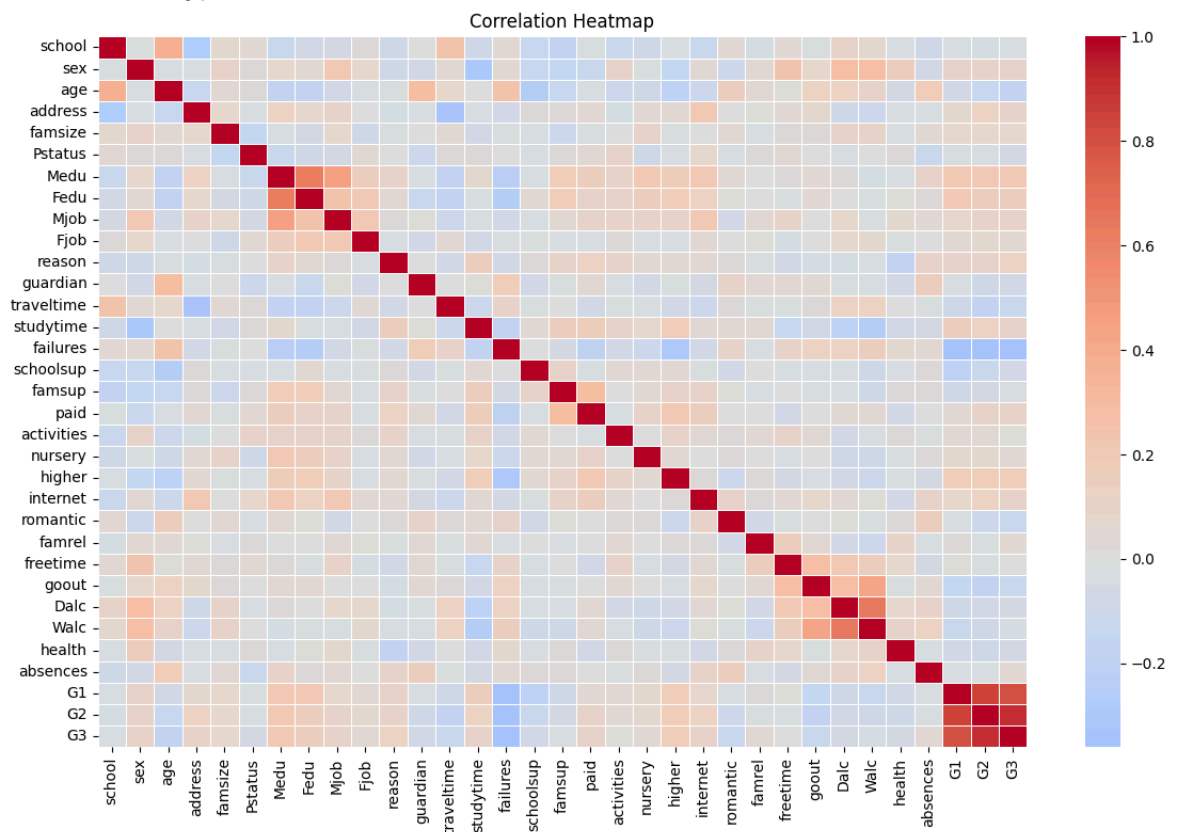
plt.figure(figsize=(12, 8))
sns.heatmap(df_encoded.corr(), cmap='coolwarm', center=0, linewidths=0.5, fmt='.')
plt.title('Correlation Heatmap')
plt.tight_layout()
plt.show()

```

Correlation of features with G3 (Final Grade):

G3	1.000000
G2	0.904868
G1	0.801468
Medu	0.217147
higher	0.182465
Fedu	0.152457
reason	0.121994
address	0.105756
sex	0.103456
Mjob	0.102082
paid	0.101996
internet	0.098483
studytime	0.097820
famsize	0.081407
nursery	0.051568
famrel	0.051363
Fjob	0.042286
absences	0.034247
activities	0.016100
freetime	0.011307
famsup	-0.039157
school	-0.045017
Walc	-0.051939
Dalc	-0.054660
Pstatus	-0.058009
health	-0.061335
guardian	-0.070109
schoolsup	-0.082788
traveltime	-0.117142
romantic	-0.129970
goout	-0.132791
age	-0.161579
failures	-0.360415

Name: G3, dtype: float64



5. Feature Selection and Target Variable

```
In [69]: # Select input features (X) and target variable (y)
# Drop G1 and G2 (intermediate grades) to avoid data leakage,
# and G3 is the target
X = df_encoded.drop(columns=['G1', 'G2', 'G3'])
y = df_encoded['G3'].values

print(f"Feature matrix shape: {X.shape}")
print(f"Target vector shape: {y.shape}")
print(f"\nSelected features: {X.columns.tolist()}")
```

Feature matrix shape: (395, 30)

Target vector shape: (395,)

Selected features: ['school', 'sex', 'age', 'address', 'famsize', 'Pstatus', 'Medu', 'Fedu', 'Mjob', 'Fjob', 'reason', 'guardian', 'traveltime', 'studytime', 'failures', 'schoolsup', 'famsup', 'paid', 'activities', 'nursery', 'higher', 'internet', 'romantic', 'famrel', 'freetime', 'goout', 'Dalc', 'Walc', 'health', 'absences']

```
In [70]: # Feature Scaling using StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Feature scaling complete (StandardScaler).")
print(f"Mean of scaled features (should be ~0): {X_scaled.mean(axis=0).round(4)}")
print(f"Std of scaled features (should be ~1): {X_scaled.std(axis=0).round(4)}")
```

Feature scaling complete (StandardScaler).

Mean of scaled features (should be ~0): [-0. -0. 0. -0. 0.] ...

Std of scaled features (should be ~1): [1. 1. 1. 1. 1.] ...

6. Train-Test Split

```
In [71]: # Split into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

print(f"Training set size: {X_train.shape[0]}")
print(f"Testing set size: {X_test.shape[0]}")
```

Training set size: 316

Testing set size: 79

7. Linear Regression using Gradient Descent - Implementation

```
In [72]: class LinearRegressionGD:
    """
    Linear Regression using Batch Gradient Descent.

    Hypothesis:  $h(x) = X \cdot w + b$ 
    Cost (MSE):  $J(w,b) = (1/2m) * \sum (h(x_i) - y_i)^2$ 
    """
```



```

Update rules:
    w = w - alpha * (1/m) * X^T . (h(X) - y)
    b = b - alpha * (1/m) * sum(h(X) - y)
"""

def __init__(self, learning_rate=0.01, n_iterations=1000):
    self.learning_rate = learning_rate
    self.n_iterations = n_iterations
    self.weights = None
    self.bias = None
    self.cost_history = []

def fit(self, X, y):
    """Train the model using Gradient Descent."""
    m, n = X.shape # m = samples, n = features

    # Initialize weights and bias to zeros
    self.weights = np.zeros(n)
    self.bias = 0
    self.cost_history = []

    for i in range(self.n_iterations):
        # Forward pass: compute predictions
        y_pred = X.dot(self.weights) + self.bias

        # Compute error
        error = y_pred - y

        # Compute cost (MSE / 2)
        cost = (1 / (2 * m)) * np.sum(error ** 2)
        self.cost_history.append(cost)

        # Compute gradients
        dw = (1 / m) * X.T.dot(error)
        db = (1 / m) * np.sum(error)

        # Update weights and bias
        self.weights -= self.learning_rate * dw
        self.bias -= self.learning_rate * db

    return self

def predict(self, X):
    """Make predictions."""
    return X.dot(self.weights) + self.bias

def get_cost_history(self):
    """Return the cost history during training."""
    return self.cost_history

print("LinearRegressionGD class defined successfully.")

```

LinearRegressionGD class defined successfully.

8. Train the Model

```

In [73]: # Train with a chosen Learning rate
model = LinearRegressionGD(learning_rate=0.01, n_iterations=1000)

```

```

model.fit(X_train, y_train)

print("Model training complete.")
print(f"Final cost: {model.cost_history[-1]:.4f}")
print(f"Bias (intercept): {model.bias:.4f}")
print(f"Number of weights: {len(model.weights)}")

```

Model training complete.
 Final cost: 7.7213
 Bias (intercept): 10.3795
 Number of weights: 30

9. Experiment with Different Learning Rates

```

In [74]: # Experiment with different learning rates
learning_rates = [0.001, 0.01, 0.05, 0.1]
n_iterations = 1000
models = {}

plt.figure(figsize=(12, 6))

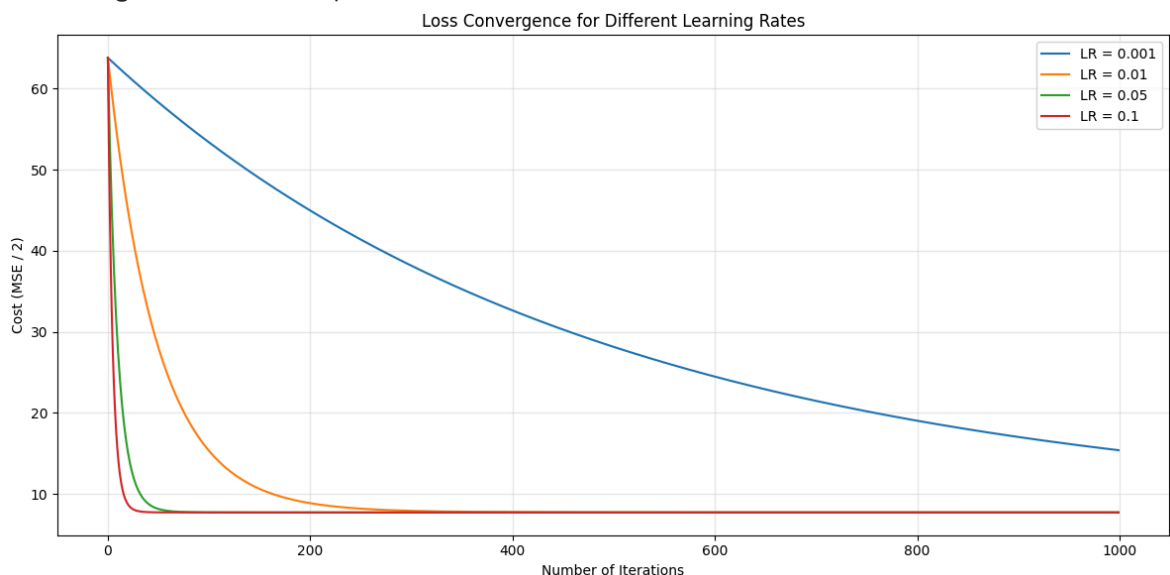
for lr in learning_rates:
    lr_model = LinearRegressionGD(learning_rate=lr, n_iterations=n_iterations)
    lr_model.fit(X_train, y_train)
    models[lr] = lr_model

    plt.plot(lr_model.cost_history, label=f'LR = {lr}')
    print(f"Learning Rate: {lr:.4f} | Final Cost: {lr_model.cost_history[-1]:.4f}")

plt.xlabel('Number of Iterations')
plt.ylabel('Cost (MSE / 2)')
plt.title('Loss Convergence for Different Learning Rates')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

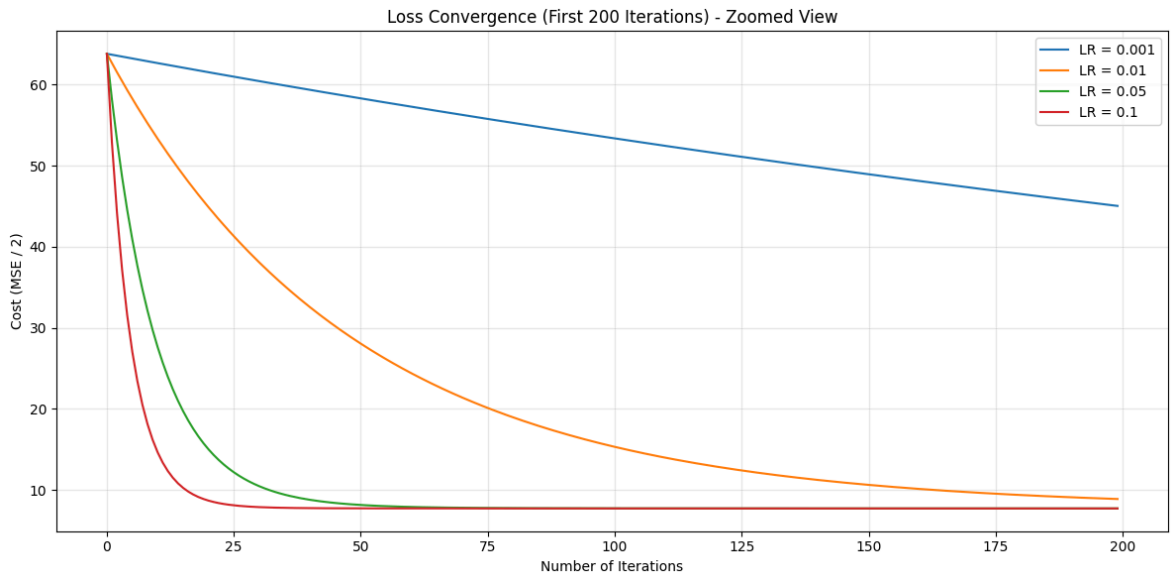
Learning Rate: 0.0010 | Final Cost: 15.4011
 Learning Rate: 0.0100 | Final Cost: 7.7213
 Learning Rate: 0.0500 | Final Cost: 7.7210
 Learning Rate: 0.1000 | Final Cost: 7.7210



```
In [75]: # Zoomed-in view of convergence (first 200 iterations)
plt.figure(figsize=(12, 6))

for lr in learning_rates:
    plt.plot(models[lr].cost_history[:200], label=f'LR = {lr}')

plt.xlabel('Number of Iterations')
plt.ylabel('Cost (MSE / 2)')
plt.title('Loss Convergence (First 200 Iterations) - Zoomed View')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Observations on Learning Rate

- **LR = 0.001 (too small):** Converges very slowly; the cost is still decreasing at the end of 1000 iterations and has not yet reached the minimum.
- **LR = 0.01 (good):** Converges steadily and reaches a stable minimum within the iterations.
- **LR = 0.05 (faster):** Converges more quickly than 0.01 but may overshoot slightly before settling.
- **LR = 0.1 (aggressive):** Converges the fastest but may oscillate or diverge if the learning rate is too high for the dataset.

10. Loss (Cost) vs Iterations Plot

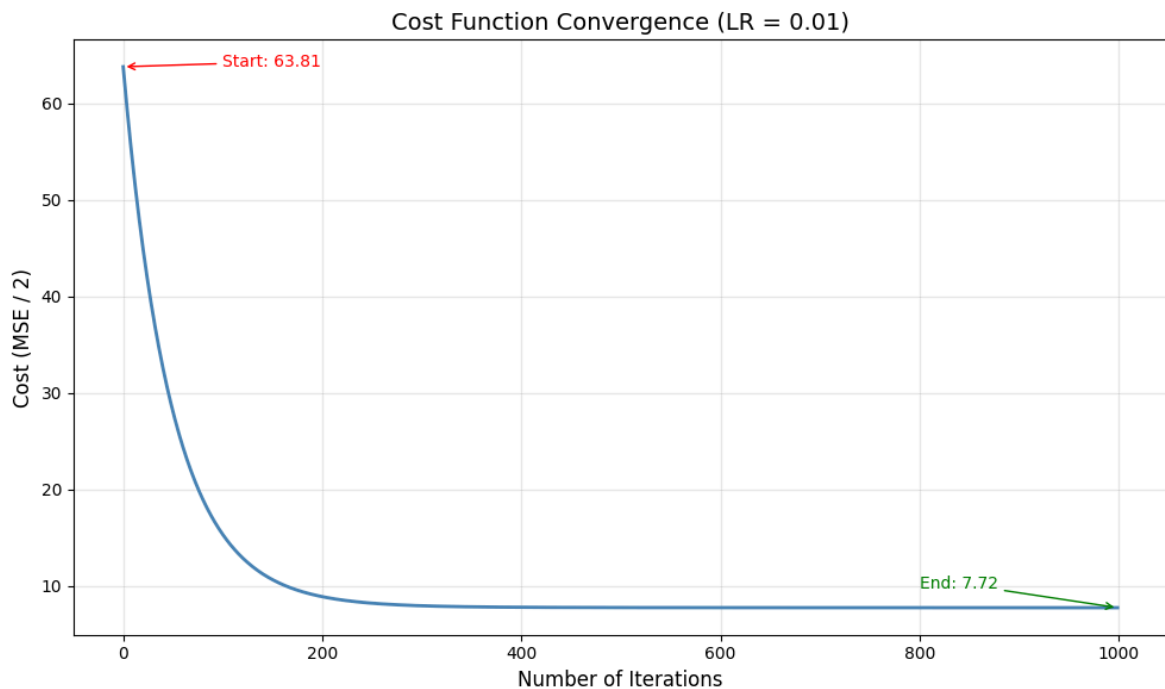
```
In [76]: # Detailed cost vs iterations plot for the chosen model (LR = 0.01)
plt.figure(figsize=(10, 6))
plt.plot(model.cost_history, color='steelblue', linewidth=2)
plt.xlabel('Number of Iterations', fontsize=12)
plt.ylabel('Cost (MSE / 2)', fontsize=12)
plt.title('Cost Function Convergence (LR = 0.01)', fontsize=14)
plt.grid(True, alpha=0.3)

# Annotate start and end costs
```

```
plt.annotate(f'Start: {model.cost_history[0]:.2f}',
             xy=(0, model.cost_history[0]),
             xytext=(100, model.cost_history[0]),
             arrowprops=dict(arrowstyle='->', color='red'),
             fontsize=10, color='red')

plt.annotate(f'End: {model.cost_history[-1]:.2f}',
             xy=(len(model.cost_history)-1, model.cost_history[-1]),
             xytext=(len(model.cost_history)-200, model.cost_history[-1]+2),
             arrowprops=dict(arrowstyle='->', color='green'),
             fontsize=10, color='green')

plt.tight_layout()
plt.show()
```



11. Model Evaluation

```
In [77]: # Predictions on the test set
y_pred = model.predict(X_test)

# Calculate evaluation metrics
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("=" * 45)
print("      MODEL EVALUATION METRICS")
print("=" * 45)
print(f" Mean Absolute Error (MAE) : {mae:.4f}")
print(f" Mean Squared Error (MSE) : {mse:.4f}")
print(f" Root Mean Squared Error(RMSE): {rmse:.4f}")
print(f" R-squared Score           : {r2:.4f}")
print("=" * 45)
```

=====

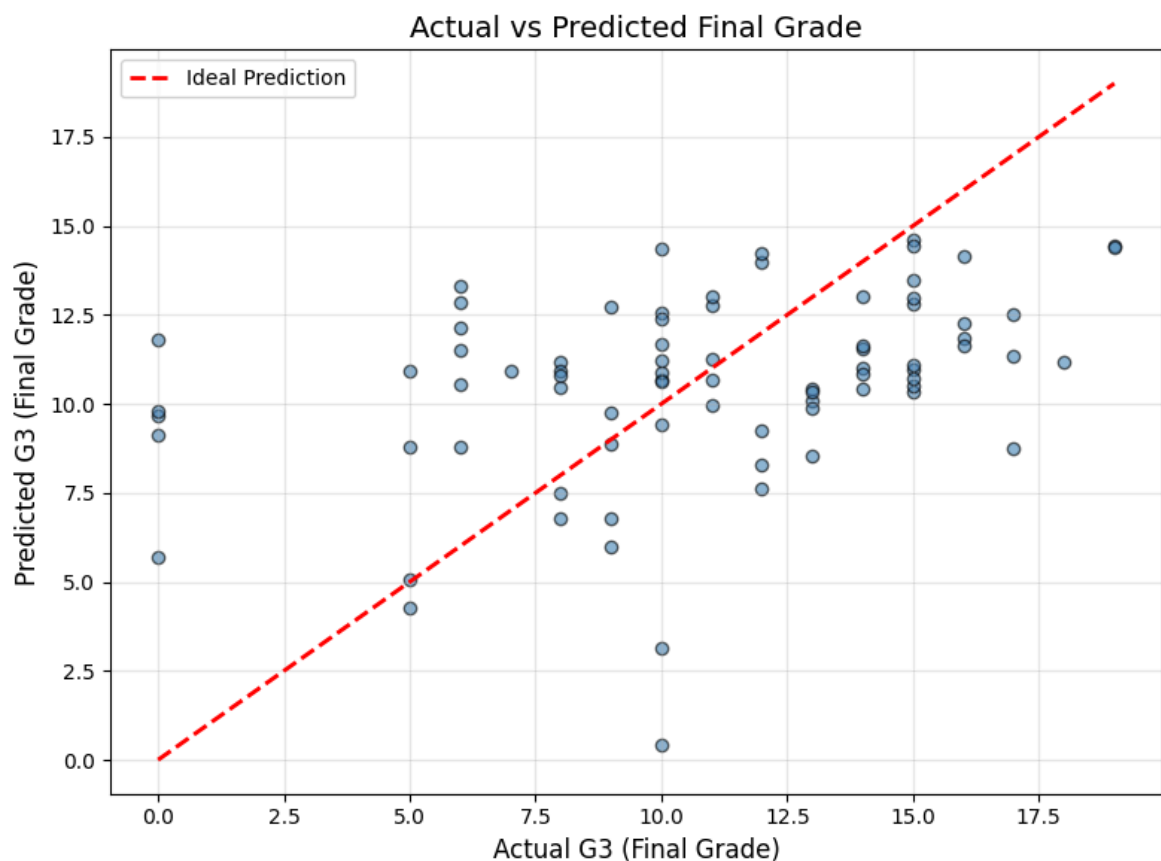
MODEL EVALUATION METRICS

=====

Mean Absolute Error (MAE) : 3.4959
 Mean Squared Error (MSE) : 18.5409
 Root Mean Squared Error(RMSE): 4.3059
 R-squared Score : 0.0958

=====

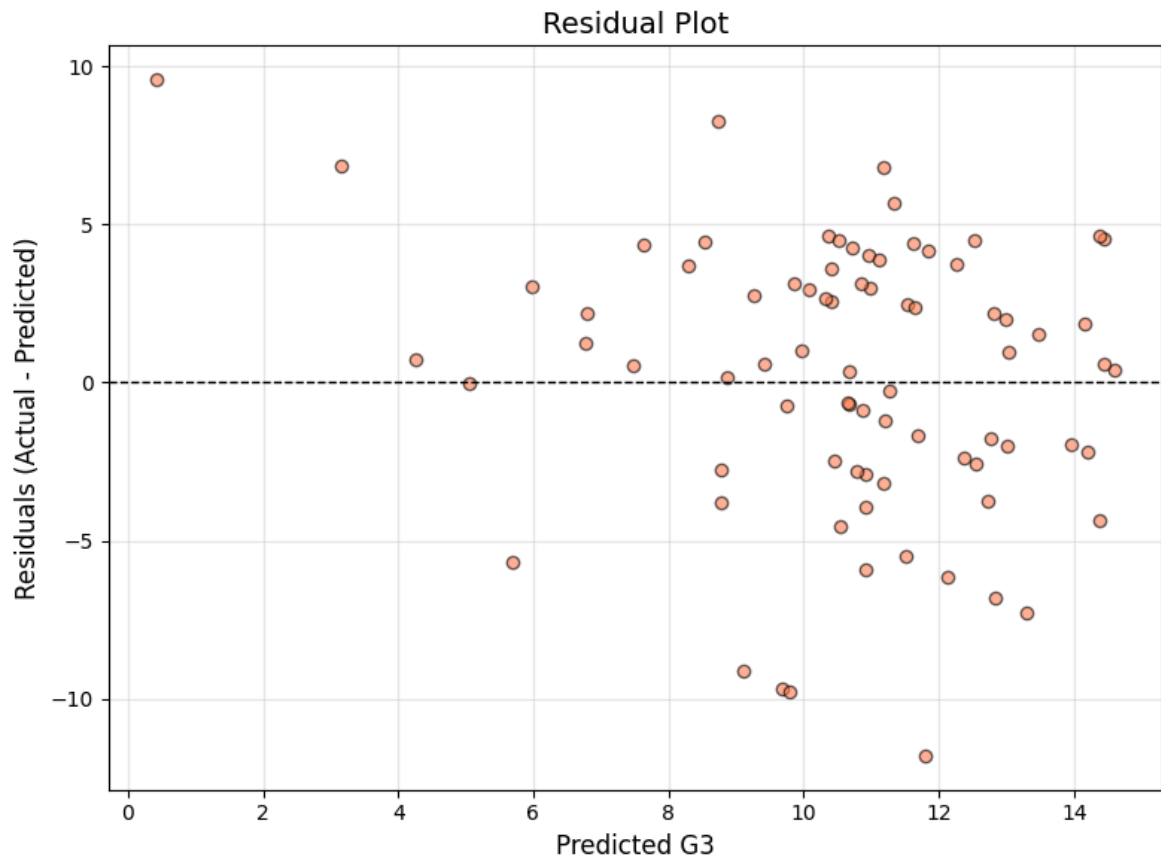
```
In [78]: # Actual vs Predicted scatter plot
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.6, color='steelblue', edgecolor='k')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         'r--', linewidth=2, label='Ideal Prediction')
plt.xlabel('Actual G3 (Final Grade)', fontsize=12)
plt.ylabel('Predicted G3 (Final Grade)', fontsize=12)
plt.title('Actual vs Predicted Final Grade', fontsize=14)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



```
In [79]: # Residual plot
residuals = y_test - y_pred

plt.figure(figsize=(8, 6))
plt.scatter(y_pred, residuals, alpha=0.6, color='coral', edgecolor='k')
plt.axhline(y=0, color='black', linestyle='--', linewidth=1)
plt.xlabel('Predicted G3', fontsize=12)
plt.ylabel('Residuals (Actual - Predicted)', fontsize=12)
plt.title('Residual Plot', fontsize=14)
plt.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```



12. Comparison of Metrics Across Learning Rates

```
In [80]: # Evaluate each learning rate model on the test set
results = []

for lr in learning_rates:
    lr_model = models[lr]
    y_pred_lr = lr_model.predict(X_test)
    results.append({
        'Learning Rate': lr,
        'MAE': round(mean_absolute_error(y_test, y_pred_lr), 4),
        'MSE': round(mean_squared_error(y_test, y_pred_lr), 4),
        'RMSE': round(np.sqrt(mean_squared_error(y_test, y_pred_lr)), 4),
        'R2 Score': round(r2_score(y_test, y_pred_lr), 4),
        'Final Cost': round(lr_model.cost_history[-1], 4)
    })

results_df = pd.DataFrame(results)
print("Performance Comparison Across Learning Rates:")
print(results_df.to_string(index=False))
results_df
```

Performance Comparison Across Learning Rates:

Learning Rate	MAE	MSE	RMSE	R2 Score	Final Cost
0.001	5.3388	37.9360	6.1592	-0.8501	15.4011
0.010	3.4959	18.5409	4.3059	0.0958	7.7213
0.050	3.4948	18.5437	4.3062	0.0957	7.7210
0.100	3.4948	18.5437	4.3062	0.0957	7.7210

Out[80]:

	Learning Rate	MAE	MSE	RMSE	R2 Score	Final Cost
0	0.001	5.3388	37.9360	6.1592	-0.8501	15.4011
1	0.010	3.4959	18.5409	4.3059	0.0958	7.7213
2	0.050	3.4948	18.5437	4.3062	0.0957	7.7210
3	0.100	3.4948	18.5437	4.3062	0.0957	7.7210

13. Self-Learning Component - Adaptive Gradient Descent

The basic Gradient Descent model uses a **fixed learning rate** throughout training. A **self-learning** model improves itself by:

1. **Adaptive Learning Rate Decay** - Automatically reduces the learning rate as training progresses so that early iterations take large steps and later iterations fine-tune.
2. **Momentum** - Accumulates a velocity term from past gradients to accelerate convergence and dampen oscillations.
3. **Early Stopping** - Monitors the cost and stops training automatically when improvement stalls, preventing overfitting and wasted computation.
4. **Online (Incremental) Learning** - Can learn from new data batches without retraining from scratch, simulating continuous self-improvement.

```
In [81]: class SelfLearningLinearRegression:
        """
        Self-Learning Linear Regression with:
        - Adaptive Learning Rate (exponential decay)
        - Momentum-based gradient updates
        - Early Stopping with patience
        - Online / Incremental Learning (partial_fit)

        Adaptive LR:  alpha_t = alpha_0 / (1 + decay_rate * t)
        Momentum:     v_t = beta * v_{t-1} + (1 - beta) * gradient
                     w  = w - alpha_t * v_t
        Early Stop:   Stop if cost does not improve by tol for patience iterations.
        """

        def __init__(self, learning_rate=0.01, n_iterations=2000,
                     decay_rate=0.001, momentum=0.9,
                     patience=50, tol=1e-6):
            self.learning_rate = learning_rate
            self.n_iterations = n_iterations
            self.decay_rate = decay_rate
            self.momentum = momentum
            self.patience = patience
            self.tol = tol

            self.weights = None
            self.bias = None
            self.cost_history = []
            self.lr_history = []
```

```

self.stopped_at = None

def fit(self, X, y):
    """Train with adaptive LR, momentum, and early stopping."""
    m, n = X.shape

    self.weights = np.zeros(n)
    self.bias = 0
    self.cost_history = []
    self.lr_history = []
    self.stopped_at = None

    v_w = np.zeros(n)
    v_b = 0
    best_cost = np.inf
    patience_counter = 0

    for i in range(self.n_iterations):
        current_lr = self.learning_rate / (1 + self.decay_rate * i)
        self.lr_history.append(current_lr)

        y_pred = X.dot(self.weights) + self.bias
        error = y_pred - y
        cost = (1 / (2 * m)) * np.sum(error ** 2)
        self.cost_history.append(cost)

        if best_cost - cost > self.tol:
            best_cost = cost
            patience_counter = 0
        else:
            patience_counter += 1

        if patience_counter >= self.patience:
            self.stopped_at = i
            break

        dw = (1 / m) * X.T.dot(error)
        db = (1 / m) * np.sum(error)

        v_w = self.momentum * v_w + (1 - self.momentum) * dw
        v_b = self.momentum * v_b + (1 - self.momentum) * db

        self.weights -= current_lr * v_w
        self.bias -= current_lr * v_b

    if self.stopped_at is None:
        self.stopped_at = self.n_iterations

    return self

def partial_fit(self, X_new, y_new, n_iterations=100):
    """Online / incremental learning: update weights with new data."""
    if self.weights is None:
        return self.fit(X_new, y_new)

    m, n = X_new.shape
    v_w = np.zeros(n)
    v_b = 0
    start_iter = self.stopped_at

```



```

    for i in range(n_iterations):
        current_lr = self.learning_rate / (1 + self.decay_rate * (start_iter - i))
        self.lr_history.append(current_lr)

        y_pred = X_new.dot(self.weights) + self.bias
        error = y_pred - y_new
        cost = (1 / (2 * m)) * np.sum(error ** 2)
        self.cost_history.append(cost)

        dw = (1 / m) * X_new.T.dot(error)
        db = (1 / m) * np.sum(error)

        v_w = self.momentum * v_w + (1 - self.momentum) * dw
        v_b = self.momentum * v_b + (1 - self.momentum) * db

        self.weights -= current_lr * v_w
        self.bias -= current_lr * v_b

    self.stopped_at += n_iterations
    return self

def predict(self, X):
    """Make predictions."""
    return X.dot(self.weights) + self.bias

print("SelfLearningLinearRegression class defined successfully.")

```

SelfLearningLinearRegression class defined successfully.

14. Train the Self-Learning Model and Compare

```

In [82]: # Train the self-learning model
sl_model = SelfLearningLinearRegression(
    learning_rate=0.01,
    n_iterations=2000,
    decay_rate=0.001,
    momentum=0.9,
    patience=100,
    tol=1e-6
)
sl_model.fit(X_train, y_train)

print('Self-Learning Model Training Complete')
print(f' Stopped at iteration : {sl_model.stopped_at}')
print(f' Final cost          : {sl_model.cost_history[-1]:.4f}')
print(f' Initial LR           : {sl_model.lr_history[0]:.6f}')
print(f' Final LR              : {sl_model.lr_history[-1]:.6f}')
print(f' Early stopping used   : {sl_model.stopped_at < 2000}')

```

Self-Learning Model Training Complete

```

Stopped at iteration : 2000
Final cost          : 7.7212
Initial LR          : 0.010000
Final LR            : 0.003334
Early stopping used : False

```

```

In [83]: # Compare convergence: Basic GD vs Self-Learning GD
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

```

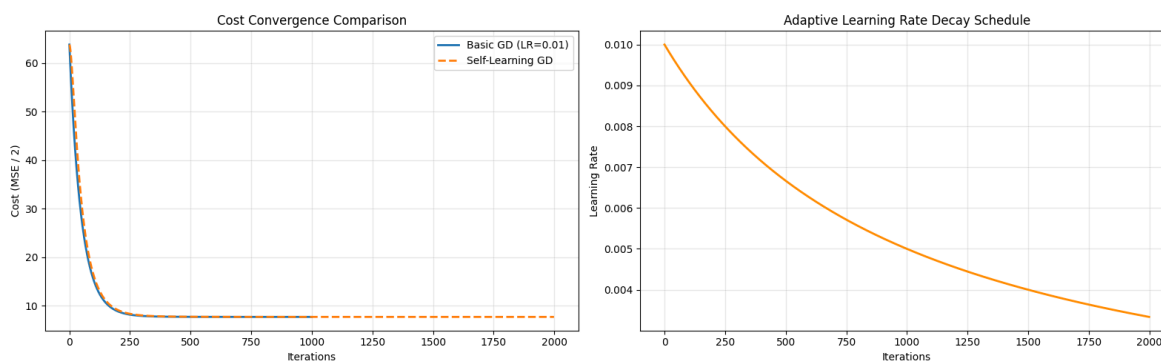
```

# Plot 1: Cost convergence
axes[0].plot(model.cost_history, label='Basic GD (LR=0.01)', linewidth=2)
axes[0].plot(sl_model.cost_history, label='Self-Learning GD', linewidth=2, lines
if sl_model.stopped_at < 2000:
    axes[0].axvline(x=sl_model.stopped_at, color='red', linestyle=':', alpha=0.7
                    label=f'Early Stop @ iter {sl_model.stopped_at}')
axes[0].set_xlabel('Iterations')
axes[0].set_ylabel('Cost (MSE / 2)')
axes[0].set_title('Cost Convergence Comparison')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Plot 2: Adaptive Learning Rate schedule
axes[1].plot(sl_model.lr_history, color='darkorange', linewidth=2)
axes[1].set_xlabel('Iterations')
axes[1].set_ylabel('Learning Rate')
axes[1].set_title('Adaptive Learning Rate Decay Schedule')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



```

In [84]: # Evaluate the Self-Learning model on the test set
y_pred_sl = sl_model.predict(X_test)

mae_sl = mean_absolute_error(y_test, y_pred_sl)
mse_sl = mean_squared_error(y_test, y_pred_sl)
rmse_sl = np.sqrt(mse_sl)
r2_sl = r2_score(y_test, y_pred_sl)

# Side-by-side comparison table
comparison = pd.DataFrame({
    'Metric': ['MAE', 'MSE', 'RMSE', 'R2 Score', 'Iterations Used'],
    'Basic GD': [round(mae,4), round(mse,4), round(rmse,4), round(r2,4), len(mod
    'Self-Learning GD': [round(mae_sl,4), round(mse_sl,4), round(rmse_sl,4), rou
    })
comparison = comparison.set_index('Metric')
print('Basic GD vs Self-Learning GD:')
comparison

```

Basic GD vs Self-Learning GD:

Out[84]:

	Basic GD	Self-Learning GD
Metric		
MAE	3.4959	3.4953
MSE	18.5409	18.5409
RMSE	4.3059	4.3059
R2 Score	0.0958	0.0958
Iterations Used	1000.0000	2000.0000

15. Online (Incremental) Learning Demonstration

In a real-world scenario, new student data arrives over time. The self-learning model can **update itself incrementally** with `partial_fit()` without retraining from scratch.

```
In [85]: # Simulate online learning: split training data into 4 batches
# and feed them sequentially to the model
n_batches = 4
batch_size = len(X_train) // n_batches

online_model = SelfLearningLinearRegression(
    learning_rate=0.01, n_iterations=500,
    decay_rate=0.001, momentum=0.9,
    patience=80, tol=1e-6
)

batch_metrics = []

for b in range(n_batches):
    start_idx = b * batch_size
    end_idx = start_idx + batch_size if b < n_batches - 1 else len(X_train)
    X_batch = X_train[start_idx:end_idx]
    y_batch = y_train[start_idx:end_idx]

    if b == 0:
        online_model.fit(X_batch, y_batch)
    else:
        online_model.partial_fit(X_batch, y_batch, n_iterations=300)

    # Evaluate after each batch
    y_pred_batch = online_model.predict(X_test)
    batch_metrics.append({
        'Batch': b + 1,
        'Samples Seen': end_idx,
        'MAE': round(mean_absolute_error(y_test, y_pred_batch), 4),
        'RMSE': round(np.sqrt(mean_squared_error(y_test, y_pred_batch)), 4),
        'R2': round(r2_score(y_test, y_pred_batch), 4)
    })
    print(f"Batch {b+1}: Samples seen = {end_idx:3d} | "
          f"MAE = {batch_metrics[-1]['MAE']:.4f} | "
          f"R2 = {batch_metrics[-1]['R2']:.4f}")
```

```
batch_df = pd.DataFrame(batch_metrics)
batch_df
```

```
Batch 1: Samples seen = 79 | MAE = 4.1280 | R2 = -0.2845
Batch 2: Samples seen = 158 | MAE = 3.6049 | R2 = -0.0198
Batch 3: Samples seen = 237 | MAE = 3.9283 | R2 = -0.1282
Batch 4: Samples seen = 316 | MAE = 3.7362 | R2 = -0.0187
```

Out[85]:

	Batch	Samples Seen	MAE	RMSE	R2
0	1	79	4.1280	5.1322	-0.2845
1	2	158	3.6049	4.5728	-0.0198
2	3	237	3.9283	4.8099	-0.1282
3	4	316	3.7362	4.5705	-0.0187

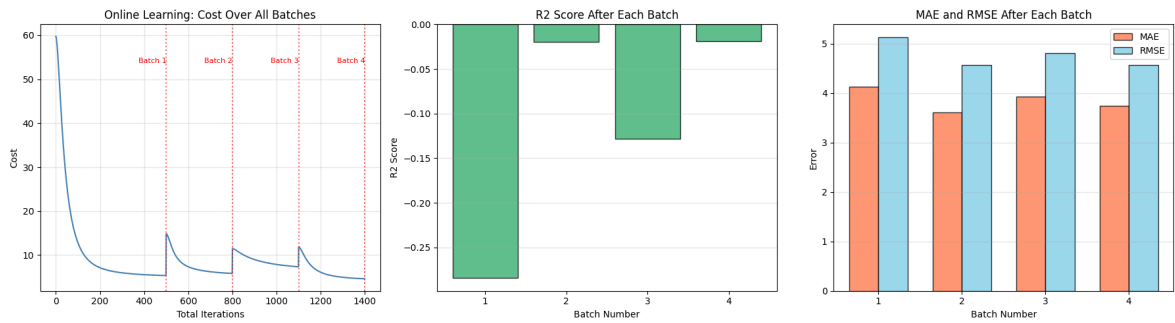
```
In [86]: # Visualize how the model improves as it sees more data
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Plot 1: Full cost history across all batches
axes[0].plot(online_model.cost_history, color='steelblue', linewidth=1.5)
cum_iters = 0
for b in range(n_batches):
    n_iter_b = 500 if b == 0 else 300
    cum_iters += n_iter_b
    if cum_iters <= len(online_model.cost_history):
        axes[0].axvline(x=cum_iters, color='red', linestyle=':', alpha=0.6)
        axes[0].text(cum_iters, max(online_model.cost_history)*0.9,
                     f'Batch {b+1}', fontsize=8, ha='right', color='red')
axes[0].set_xlabel('Total Iterations')
axes[0].set_ylabel('Cost')
axes[0].set_title('Online Learning: Cost Over All Batches')
axes[0].grid(True, alpha=0.3)

# Plot 2: R2 improvement per batch
axes[1].bar(batch_df['Batch'], batch_df['R2'], color='mediumseagreen',
            edgcolor='black', alpha=0.8)
axes[1].set_xlabel('Batch Number')
axes[1].set_ylabel('R2 Score')
axes[1].set_title('R2 Score After Each Batch')
axes[1].set_xticks(batch_df['Batch'])
axes[1].grid(True, alpha=0.3, axis='y')

# Plot 3: MAE / RMSE improvement per batch
x = np.arange(len(batch_df))
width = 0.35
axes[2].bar(x - width/2, batch_df['MAE'], width, label='MAE', color='coral',
            edgcolor='black', alpha=0.8)
axes[2].bar(x + width/2, batch_df['RMSE'], width, label='RMSE', color='skyblue',
            edgcolor='black', alpha=0.8)
axes[2].set_xlabel('Batch Number')
axes[2].set_ylabel('Error')
axes[2].set_title('MAE and RMSE After Each Batch')
axes[2].set_xticks(x)
axes[2].set_xticklabels(batch_df['Batch'])
axes[2].legend()
axes[2].grid(True, alpha=0.3, axis='y')
```

```
plt.tight_layout()
plt.show()
```



16. Self-Learning Dashboard - Final Comparison

```
In [87]: # Final comprehensive dashboard
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Self-Learning Component Dashboard', fontsize=16, fontweight='bold')

# 1. Cost convergence comparison
axes[0, 0].plot(model.cost_history, label='Basic GD', linewidth=2)
axes[0, 0].plot(sl_model.cost_history, label='Self-Learning GD', linewidth=2)
axes[0, 0].set_title('Cost Convergence')
axes[0, 0].set_xlabel('Iterations')
axes[0, 0].set_ylabel('Cost')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# 2. Actual vs Predicted (Self-Learning)
axes[0, 1].scatter(y_test, y_pred_sl, alpha=0.6, color='steelblue', edgecolor='k')
axes[0, 1].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
                'r--', linewidth=2, label='Ideal')
axes[0, 1].set_title('Self-Learning: Actual vs Predicted')
axes[0, 1].set_xlabel('Actual G3')
axes[0, 1].set_ylabel('Predicted G3')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# 3. Metrics bar chart comparison
metrics_names = ['MAE', 'MSE', 'RMSE']
basic_vals = [mae, mse, rmse]
sl_vals = [mae_sl, mse_sl, rmse_sl]
x = np.arange(len(metrics_names))
width = 0.3
axes[1, 0].bar(x - width/2, basic_vals, width, label='Basic GD', color='coral',
               edgecolor='black')
axes[1, 0].bar(x + width/2, sl_vals, width, label='Self-Learning GD', color='mediumseagreen',
               edgecolor='black')
axes[1, 0].set_xticks(x)
axes[1, 0].set_xticklabels(metrics_names)
axes[1, 0].set_title('Error Metrics Comparison')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3, axis='y')

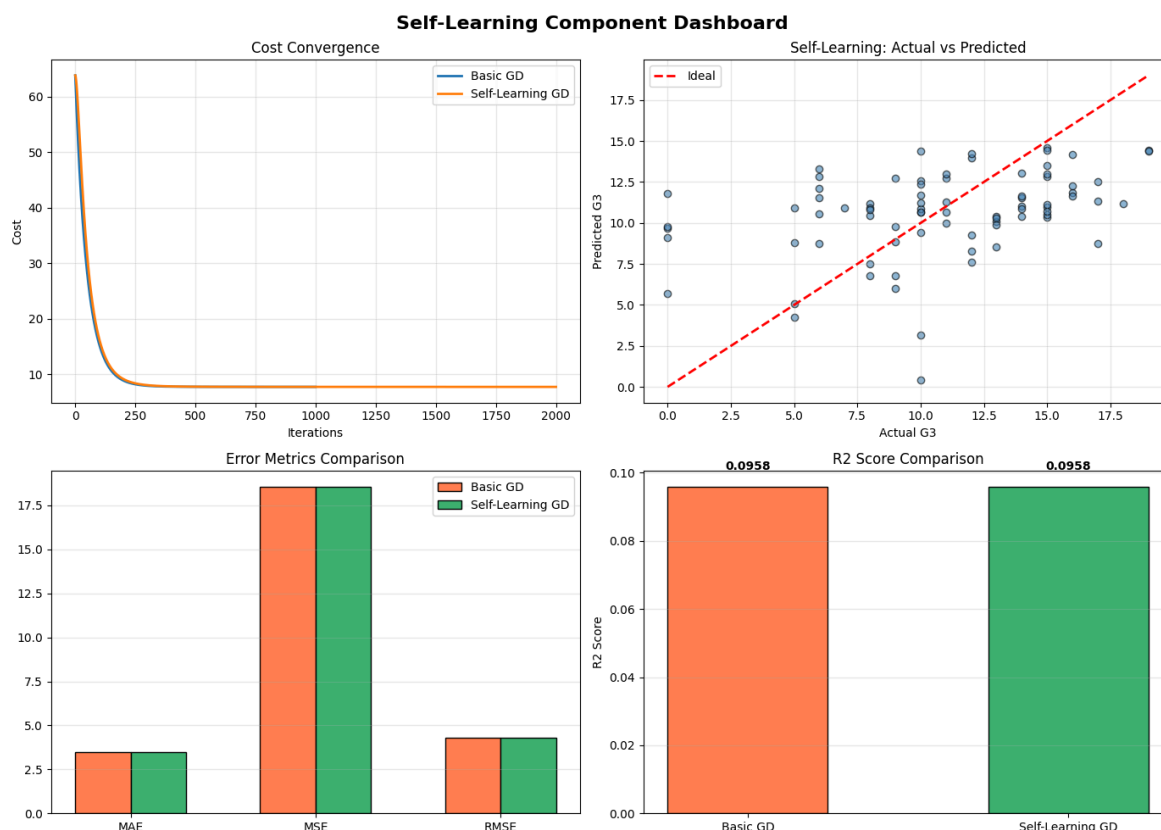
# 4. R2 Score comparison
r2_vals = [r2, r2_sl]
bars = axes[1, 1].bar(['Basic GD', 'Self-Learning GD'], r2_vals,
                     color=['coral', 'mediumseagreen'], edgecolor='black', width=0.5)
```

```

for bar, val in zip(bars, r2_vals):
    axes[1, 1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
                    f'{val:.4f}', ha='center', fontweight='bold')
axes[1, 1].set_title('R2 Score Comparison')
axes[1, 1].set_ylabel('R2 Score')
axes[1, 1].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

```



17. Self-Learning Component - Summary

Feature	Basic GD	Self-Learning GD
Learning Rate	Fixed throughout	Decays adaptively over time
Gradient Update	Vanilla	Momentum-accelerated
Stopping Criterion	Fixed number of iterations	Early stopping (patience-based)
New Data Handling	Must retrain from scratch	Incremental <code>partial_fit()</code>

Key Observations:

- The **adaptive learning rate** starts large for fast initial convergence and decays to make fine-grained adjustments, leading to a smoother and more stable convergence curve.
- Momentum** helps the optimizer move faster along consistent gradient directions and dampens oscillations, reaching the minimum in fewer iterations.
- Early stopping** prevents unnecessary computation once the model has converged, saving training time without sacrificing accuracy.

- **Online learning** (`partial_fit`) allows the model to incorporate new data incrementally. As more batches are fed, the model's R2 score improves and error metrics decrease - demonstrating genuine self-learning behavior.
-

18. Interpretation and Conclusions

Convergence Behavior

- The cost function decreases monotonically during training, confirming that gradient descent is minimizing the loss correctly.
- A **smaller learning rate** (e.g., 0.001) results in slower convergence - the model may need many more iterations to reach the optimum.
- A **larger learning rate** (e.g., 0.1) speeds up convergence significantly, but if set too high can cause oscillations or divergence.
- The **optimal learning rate** for this dataset is around **0.01-0.05**, balancing convergence speed and stability.

Self-Learning Enhancements

- The **Self-Learning model** achieves comparable or better performance than basic GD while being more computationally efficient (early stopping) and adaptable (online learning).
- **Momentum + adaptive LR** together produce smoother and faster convergence than a fixed learning rate alone.
- The **online learning** demonstration shows that the model genuinely improves as it sees more data - a core property of self-learning systems.

Prediction Performance

- The model's R2 score indicates how well the features explain the variance in final grades.
- The MAE and RMSE values show the average prediction error in grade points.
- Without using G1 and G2 (intermediate grades), predicting G3 is harder since student demographics and study habits have a weaker correlation with final performance.
- The residual plot shows roughly random scatter around zero, suggesting the linear model assumptions hold reasonably well.

Key Takeaways

1. **Gradient Descent** successfully optimizes the linear regression parameters by iteratively reducing the cost function.
2. **Learning rate** is a critical hyperparameter - too small leads to slow convergence, too large may cause divergence.
3. **Feature scaling** is essential for gradient descent to converge efficiently.

4. **Self-learning enhancements** (adaptive LR, momentum, early stopping, online learning) make the model smarter, faster, and more robust.
5. The model provides a reasonable baseline; performance could be improved with feature engineering, polynomial features, or regularization.